

当进入到循环体内部后, 程序通过某种规则移动 `beg` 或者 `end` 来缩小搜索的范围。如果 `mid` 所指的元素比要找的元素 `sought` 大, 可推测若 `text` 含有 `sought`, 则必出现在 `mid` 所指元素的前面。此时, 可以忽略 `mid` 后面的元素不再查找, 并把 `mid` 赋给 `end` 即可。另一种情况, 如果 `*mid` 比 `sought` 小, 则要找的元素必出现在 `mid` 所指元素的后面。此时, 通过令 `beg` 指向 `mid` 的下一个位置即可改变搜索范围。因为已经验证过 `mid` 不是我们要找的对象, 所以在接下来的搜索中不必考虑它。

循环过程终止时, `mid` 或者等于 `end` 或者指向要找的元素。如果 `mid` 等于 `end`, 说明 `text` 中没有我们要找的元素。

3.4.2 节练习

113

练习 3.24: 请使用迭代器重做 3.3.3 节 (第 94 页) 的最后一个练习。

练习 3.25: 3.3.3 节 (第 93 页) 划分分数段的程序是使用下标运算符实现的, 请利用迭代器改写该程序并实现完全相同的功能。

练习 3.26: 在 100 页的二分搜索程序中, 为什么用的是 `mid = beg + (end - beg) / 2`, 而非 `mid = (beg + end) / 2`?

3.5 数组

数组是一种类似于标准库类型 `vector` (参见 3.3 节, 第 86 页) 的数据结构, 但是在性能和灵活性的权衡上又与 `vector` 有所不同。与 `vector` 相似的地方是, 数组也是存放类型相同的对象的容器, 这些对象本身没有名字, 需要通过其所在位置访问。与 `vector` 不同的地方是, 数组的大小确定不变, 不能随意向数组中增加元素。因为数组的大小固定, 因此对某些特殊的应用来说程序的运行时性能较好, 但是相应地也损失了一些灵活性。



如果不清楚元素的确切个数, 请使用 `vector`。

3.5.1 定义和初始化内置数组

数组是一种复合类型 (参见 2.3 节, 第 45 页)。数组的声明形如 `a[d]`, 其中 `a` 是数组的名字, `d` 是数组的维度。维度说明了数组中元素的个数, 因此必须大于 0。数组中元素的个数也属于数组类型的一部分, 编译的时候维度应该是已知的。也就是说, 维度必须是一个常量表达式 (参见 2.4.4 节, 第 58 页):

```
unsigned cnt = 42;           // 不是常量表达式
constexpr unsigned sz = 42; // 常量表达式, 关于 constexpr, 参见 2.4.4 节 (第 59 页)
int arr[10];                 // 含有 10 个整数的数组
int *parr[sz];               // 含有 42 个整型指针的数组
string bad[cnt];             // 错误: cnt 不是常量表达式
string strs[get_size()];     // 当 get_size 是 constexpr 时正确; 否则错误
```

默认情况下, 数组的元素被默认初始化 (参见 2.2.1 节, 第 40 页)。



和内置类型的变量一样, 如果在函数内部定义了某种内置类型的数组, 那么默认初始化会令数组含有未定义的值。

定义数组时必须指定数组的类型，不允许用 `auto` 关键字由初始值的列表推断类型。另外和 `vector` 一样，数组的元素应为对象，因此不存在引用的数组。

114 显式初始化数组元素

可以对数组的元素进行列表初始化（参见 3.3.1 节，第 88 页），此时允许忽略数组的维度。如果在声明时没有指明维度，编译器会根据初始值的数量计算并推测出来；相反，如果指明了维度，那么初始值的总数量不应该超出指定的大小。如果维度比提供的初始值数量大，则用提供的初始值初始化靠前的元素，剩下的元素被初始化成默认值（参见 3.3.1 节，第 88 页）：

```
const unsigned sz = 3;
int ial[sz] = {0, 1, 2};           // 含有 3 个元素的数组, 元素值分别是 0, 1, 2
int a2[] = {0, 1, 2};             // 维度是 3 的数组
int a3[5] = {0, 1, 2};            // 等价于 a3[] = {0, 1, 2, 0, 0}
string a4[3] = {"hi", "bye"};     // 等价于 a4[] = {"hi", "bye", ""}
int a5[2] = {0, 1, 2};            // 错误: 初始值过多
```

字符数组的特殊性

字符数组有一种额外的初始化形式，我们可以用字符串字面值（参见 2.1.3 节，第 36 页）对此类数组初始化。当使用这种方式时，一定要注意字符串字面值的结尾处还有一个空字符，这个空字符也会像字符串的其他字符一样被拷贝到字符数组中去：

```
char a1[] = {'C', '+', '+'};      // 列表初始化, 没有空字符
char a2[] = {'C', '+', '+', '\0'}; // 列表初始化, 含有显式的空字符
char a3[] = "C++";               // 自动添加表示字符串结束的空字符
const char a4[6] = "Daniel";     // 错误: 没有空间可存放空字符!
```

`a1` 的维度是 3, `a2` 和 `a3` 的维度都是 4, `a4` 的定义是错误的。尽管字符串字面值 "Daniel" 看起来只有 6 个字符，但是数组的大小必须至少是 7，其中 6 个位置存放字面值的内容，另外 1 个存放结尾处的空字符。

不允许拷贝和赋值

不能将数组的内容拷贝给其他数组作为其初始值，也不能用数组为其他数组赋值：

```
int a[] = {0, 1, 2};             // 含有 3 个整数的数组
int a2[] = a;                    // 错误: 不允许使用一个数组初始化另一个数组
a2 = a;                          // 错误: 不能把一个数组直接赋值给另一个数组
```



一些编译器支持数组的赋值，这就是所谓的编译器扩展（`compiler extension`）。但一般来说，最好避免使用非标准特性，因为含有非标准特性的程序很可能在其他编译器上无法正常工作。

理解复杂的数组声明

和 `vector` 一样，数组能存放大多数类型的对象。例如，可以定义一个存放指针的数组。又因为数组本身就是对象，所以允许定义数组的指针及数组的引用。在这几种情况中，定义存放指针的数组比较简单和直接，但是定义数组的指针或数组的引用就稍微复杂一点了：

```
115 int *ptrs[10];                // ptrs 是含有 10 个整型指针的数组
int &refs[10] = /* ? */;         // 错误: 不存在引用的数组
int (*Parray)[10] = &arr;       // Parray 指向一个含有 10 个整数的数组
int (&arrRef)[10] = arr;        // arrRef 引用一个含有 10 个整数的数组
```

默认情况下，类型修饰符从右向左依次绑定。对于 `ptrs` 来说，从右向左（参见 2.3.3 节，第 52 页）理解其含义比较简单：首先知道我们定义的是一个大小为 10 的数组，它的名字是 `ptrs`，然后知道数组中存放的是指向 `int` 的指针。

但是对于 `Parray` 来说，从右向左理解就不太合理了。因为数组的维度是紧跟着被声明的名字的，所以就数组而言，由内向外阅读要比从右向左好多了。由内向外的顺序可帮助我们更好地理解 `Parray` 的含义：首先是圆括号括起来的部分，`*Parray` 意味着 `Parray` 是个指针，接下来观察右边，可知道 `Parray` 是个指向大小为 10 的数组的指针，最后观察左边，知道数组中的元素是 `int`。这样最终的含义就明白无误了，`Parray` 是一个指针，它指向一个 `int` 数组，数组中包含 10 个元素。同理，`(&arrRef)` 表示 `arrRef` 是一个引用，它引用的对象是一个大小为 10 的数组，数组中元素的类型是 `int`。

当然，对修饰符的数量并没有特殊限制：

```
int *(&array)[10] = ptrs; // array 是数组的引用，该数组含有 10 个指针
```

按照由内向外的顺序阅读上述语句，首先知道 `array` 是一个引用，然后观察右边知道，`array` 引用的对象是一个大小为 10 的数组，最后观察左边知道，数组的元素类型是指向 `int` 的指针。这样，`array` 就是一个含有 10 个 `int` 型指针的数组的引用。



要想理解数组声明的含义，最好的办法是从数组的名字开始按照由内向外的顺序阅读。

3.5.1 节练习

练习 3.27: 假设 `txt_size` 是一个无参数的函数，它的返回值是 `int`。请回答下列哪个定义是非法的？为什么？

```
unsigned buf_size = 1024;
(a) int ia[buf_size];           (b) int ia[4 * 7 - 14];
(c) int ia[txt_size()];        (d) char st[11] = "fundamental";
```

练习 3.28: 下列数组中元素的值是什么？

```
string sa[10];
int ia[10];
int main() {
    string sa2[10];
    int ia2[10];
}
```

练习 3.29: 相比于 `vector` 来说，数组有哪些缺点，请列举一些。

3.5.2 访问数组元素

116

与标准库类型 `vector` 和 `string` 一样，数组的元素也能使用范围 `for` 语句或下标运算符来访问。数组的索引从 0 开始，以一个包含 10 个元素的数组为例，它的索引从 0 到 9，而非从 1 到 10。

在使用数组下标的时候，通常将其定义为 `size_t` 类型。`size_t` 是一种机器相关的无符号类型，它被设计得足够大以便能表示内存中任意对象的大小。在 `cstdint` 头文件中定义了 `size_t` 类型，这个文件是 C 标准库 `stdint.h` 头文件的 C++ 语言版本。

数组除了大小固定这一特点外，其他用法与 `vector` 基本类似。例如，可以用数组来记录各分数段的成绩个数，从而实现与 3.3.3 节（第 93 页）的程序一样的功能：

```
// 以 10 分为一个分数段统计成绩的数量： 0~9, 10~19, ..., 90~99, 100
unsigned scores[11] = {}; // 11 个分数段，全部初始化为 0
unsigned grade;
while (cin >> grade) {
    if (grade <= 100)
        ++scores[grade/10]; // 将当前分数段的计数值加 1
}
```

与 93 页的程序相比，上面程序最大的不同是 `scores` 的声明。这里 `scores` 是一个含有 11 个无符号元素的数组。另外一处不太明显的区别是，本例所用的下标运算符是由 C++ 语言直接定义的，这个运算符能用在数组类型的运算对象上。93 页的那个程序所用的下标运算符是库模板 `vector` 定义的，只能用于 `vector` 类型的运算对象。

与 `vector` 和 `string` 一样，当需要遍历数组的所有元素时，最好的办法也是使用范围 `for` 语句。例如，下面的程序输出所有的 `scores`：

```
for (auto i : scores) // 对于 scores 中的每个计数值
    cout << i << " "; // 输出当前的计数值
cout << endl;
```

因为维度是数组类型的一部分，所以系统知道数组 `scores` 中有多少个元素，使用范围 `for` 语句可以减轻人为控制遍历过程的负担。

检查下标的值

与 `vector` 和 `string` 一样，数组的下标是否在合理范围之内由程序员负责检查，所谓合理就是说下标应该大于等于 0 而且小于数组的大小。要想防止数组下标越界，除了小心谨慎注意细节以及对代码进行彻底的测试之外，没有其他好办法。对于一个程序来说，即使顺利通过编译并执行，也不能肯定它不包含此类致命的错误。



大多数常见的安全问题都源于缓冲区溢出错。当数组或其他类似数据结构的下标越界并试图访问非法内存区域时，就会产生此类错误。

117

3.5.2 节练习

练习 3.30：指出下面代码中的索引错误。

```
constexpr size_t array_size = 10;
int ia[array_size];
for (size_t ix = 1; ix <= array_size; ++ix)
    ia[ix] = ix;
```

练习 3.31：编写一段程序，定义一个含有 10 个 `int` 的数组，令每个元素的值就是其下标值。

练习 3.32：将上一题刚刚创建的数组拷贝给另外一个数组。利用 `vector` 重写程序，实现类似的功能。

练习 3.33：对于 104 页的程序来说，如果不初始化 `scores` 将发生什么？

3.5.3 指针和数组

在 C++ 语言中, 指针和数组有非常紧密的联系。就如即将介绍的, 使用数组的时候编译器一般会把它转换成指针。

通常情况下, 使用取地址符 (参见 2.3.2 节, 第 47 页) 来获取指向某个对象的指针, 取地址符可以用于任何对象。数组的元素也是对象, 对数组使用下标运算符得到该数组指定位置的元素。因此像其他对象一样, 对数组的元素使用取地址符就能得到指向该元素的指针:

```
string nums[] = {"one", "two", "three"}; // 数组的元素是 string 对象
string *p = &nums[0];                  // p 指向 nums 的第一个元素
```

然而, 数组还有一个特性: 在很多用到数组名字的地方, 编译器都会自动地将其替换为一个指向数组首元素的指针:

```
string *p2 = nums; // 等价于 p2 = &nums[0]
```



在大多数表达式中, 使用数组类型的对象其实是使用一个指向该数组首元素的指针。

由上可知, 在一些情况下数组的操作实际上是指针的操作, 这一结论有很多隐含的意思。其中一层意思是当使用数组作为一个 auto (参见 2.5.2 节, 第 61 页) 变量的初始值时, 推断得到的类型是指针而非数组:

```
int ia[] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}; // ia 是一个含有 10 个整数的数组
auto ia2(ia); // ia2 是一个整型指针, 指向 ia 的第一个元素
ia2 = 42; // 错误: ia2 是一个指针, 不能用 int 值给指针赋值
```

尽管 ia 是由 10 个整数构成的数组, 但当使用 ia 作为初始值时, 编译器实际执行的初始化过程类似于下面的形式:

```
auto ia2(&ia[0]); // 显然 ia2 的类型是 int*
```

118

必须指出的是, 当使用 decltype 关键字 (参见 2.5.3 节, 第 62 页) 时上述转换不会发生, decltype(ia) 返回的类型是由 10 个整数构成的数组:

```
// ia3 是一个含有 10 个整数的数组
decltype(ia) ia3 = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
ia3 = p; // 错误: 不能用整型指针给数组赋值
ia3[4] = i; // 正确: 把 i 的值赋给 ia3 的一个元素
```

指针也是迭代器

与 2.3.2 节 (第 47 页) 介绍的内容相比, 指向数组元素的指针拥有更多功能。vector 和 string 的迭代器 (参见 3.4 节, 第 95 页) 支持的运算, 数组的指针全都支持。例如, 允许使用递增运算符将指向数组元素的指针向前移动到下一个位置上:

```
int arr[] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
int *p = arr; // p 指向 arr 的第一个元素
++p; // p 指向 arr[1]
```

就像使用迭代器遍历 vector 对象中的元素一样, 使用指针也能遍历数组中的元素。当然, 这样做的前提是先得获取到指向数组第一个元素的指针和指向数组尾元素的下一位置的指针。之前已经介绍过, 通过数组名字或者数组中首元素的地址都能得到指向首元素的指针; 不过获取尾后指针就要用到数组的另外一个特殊性质了。我们可以设法获取数组

尾元素之后的那个并不存在的元素的地址：

```
int *e = &arr[10]; // 指向 arr 尾元素的下一位置的指针
```

这里显然使用下标运算符索引了一个不存在的元素，arr 有 10 个元素，尾元素所在位置的索引是 9，接下来那个不存在的元素唯一的用处就是提供其地址用于初始化 e。就像尾后迭代器（参见 3.4.1 节，第 95 页）一样，尾后指针也不指向具体的元素。因此，不能对尾后指针执行解引用或递增的操作。

利用上面得到的指针能重写之前的循环，令其输出 arr 的全部元素：

```
for (int *b = arr; b != e; ++b)
    cout << *b << endl; // 输出 arr 的元素
```

标准库函数 begin 和 end

尽管能计算得到尾后指针，但这种用法极易出错。为了让指针的使用更简单、更安全，C++11 新标准引入了两个名为 begin 和 end 的函数。这两个函数与容器中的两个同名成员（参见 3.4.1 节，第 95 页）功能类似，不过数组毕竟不是类类型，因此这两个函数不是成员函数。正确的使用形式是将数组作为它们的参数：

C++
11

```
int ia[] = {0,1,2,3,4,5,6,7,8,9}; // ia 是一个含有 10 个整数的数组
int *beg = begin(ia);             // 指向 ia 首元素的指针
int *last = end(ia);              // 指向 arr 尾元素的下一位置的指针
```

119 begin 函数返回指向 ia 首元素的指针，end 函数返回指向 ia 尾元素下一位置的指针，这两个函数定义在 iterator 头文件中。

使用 begin 和 end 可以很容易地写出一个循环并处理数组中的元素。例如，假设 arr 是一个整型数组，下面的程序负责找到 arr 中的第一个负数：

```
// pbeg 指向 arr 的首元素，pend 指向 arr 尾元素的下一位置
int *pbeg = begin(arr), *pend = end(arr);
// 寻找第一个负值元素，如果已经检查完全部元素则结束循环
while (pbeg != pend && *pbeg >= 0)
    ++pbeg;
```

首先定义了两个名为 pbeg 和 pend 的整型指针，其中 pbeg 指向 arr 的第一个元素，pend 指向 arr 尾元素的下一位置。while 语句的条件部分通过比较 pbeg 和 pend 来确保可以安全地对 pbeg 解引用，如果 pbeg 确实指向了一个元素，将其解引用并检查元素值是否为负值。如果是，条件失效、退出循环；如果不是，将指针向前移动一位继续考查下一个元素。



一个指针如果指向了某种内置类型数组的尾元素的“下一位置”，则其具备与 vector 的 end 函数返回的与迭代器类似的功能。特别要注意，尾后指针不能执行解引用和递增操作。

指针运算

指向数组元素的指针可以执行表 3.6（第 96 页）和表 3.7（第 99 页）列出的所有迭代器运算。这些运算，包括解引用、递增、比较、与整数相加、两个指针相减等，用在指针和用在迭代器上意义完全一致。

给（从）一个指针加上（减去）某整数值，结果仍是指针。新指针指向的元素与原来

的指针相比前进了（后退了）该整数值个位置：

```
constexpr size_t sz = 5;
int arr[sz] = {1,2,3,4,5};
int *ip = arr;          // 等价于 int *ip = &arr[0]
int *ip2 = ip + 4;      // ip2 指向 arr 的尾元素 arr[4]
```

ip 加上 4 所得的结果仍是一个指针，该指针所指的元素与 ip 原来所指的元素相比前进了 4 个位置。

给指针加上一个整数，得到的新指针仍需指向同一数组的其他元素，或者指向同一数组的尾元素的下一位置：

```
// 正确：arr 转换成指向它首元素的指针；p 指向 arr 尾元素的下一位置
int *p = arr + sz;          // 使用警告：不要解引用！
int *p2 = arr + 10;        // 错误：arr 只有 5 个元素，p2 的值未定义
```

当给 arr 加上 sz 时，编译器自动地将 arr 转换成指向数组 arr 中首元素的指针。执行加法后，指针从首元素开始向前移动了 sz（这里是 5）个位置，指向新位置的元素。也就是说，它指向了数组 arr 尾元素的下一位置。如果计算所得的指针超出了上述范围就将产生错误，而且这种错误编译器一般发现不了。

120

和迭代器一样，两个指针相减的结果是它们之间的距离。参与运算的两个指针必须指向同一个数组当中的元素：

```
auto n = end(arr) - begin(arr); // n 的值是 5，也就是 arr 中元素的数量
```

两个指针相减的结果的类型是一种名为 **ptrdiff_t** 的标准库类型，和 **size_t** 一样，**ptrdiff_t** 也是一种定义在 **cstdint** 头文件中的机器相关的类型。因为差值可能为负值，所以 **ptrdiff_t** 是一种带符号类型。

只要两个指针指向同一个数组的元素，或者指向该数组的尾元素的下一位置，就能利用关系运算符对其进行比较。例如，可以按照如下的方式遍历数组中的元素：

```
int *b = arr, *e = arr + sz;
while (b < e) {
    // 使用 *b
    ++b;
}
```

如果两个指针分别指向不相关的对象，则不能比较它们：

```
int i = 0, sz = 42;
int *p = &i, *e = &sz;
// 未定义的：p 和 e 无关，因此比较毫无意义！
while (p < e)
```

尽管作用可能不是特别明显，但必须说明的是，上述指针运算同样适用于空指针（参见 2.3.2 节，第 48 页）和所指对象并非数组的指针。在后一种情况下，两个指针必须指向同一个对象或该对象的下一位置。如果 p 是空指针，允许给 p 加上或减去一个值为 0 的整型常量表达式（参见 2.4.4 节，第 58 页）。两个空指针也允许彼此相减，结果当然是 0。

解引用和指针运算的交互

指针加上一个整数所得的结果还是一个指针。假设结果指针指向了一个元素，则允许解引用该结果指针：

```
int ia[] = {0,2,4,6,8}; // 含有5个整数的数组
int last = *(ia + 4);    // 正确: 把 last 初始化成 8, 也就是 ia[4] 的值
```

表达式 $*(ia+4)$ 计算 ia 前进 4 个元素后的新地址, 解引用该结果指针的效果等价于表达式 $ia[4]$ 。

回忆一下在 3.4.1 节 (第 98 页) 中介绍过如果表达式含有解引用运算符和点运算符, 最好在必要的地方加上圆括号。类似的, 此例中指针加法的圆括号也不可缺少。如果写成下面的形式:

```
last = *ia + 4; // 正确: last = 4 等价于 ia[0] + 4
```

含义就与之前完全不同了, 此时先解引用 ia , 然后给解引用的结果再加上 4。4.1.2 节 (第 121 页) 将对这一问题做进一步分析。



下标和指针

121

如前所述, 在很多情况下使用数组的名字其实用的是一个指向数组首元素的指针。一个典型的例子是当对数组使用下标运算符时, 编译器会自动执行上述转换操作。给定

```
int ia[] = {0,2,4,6,8}; // 含有5个整数的数组
```

此时, $ia[0]$ 是一个使用了数组名字的表达式, 对数组执行下标运算其实是对指向数组元素的指针执行下标运算:

```
int i = ia[2];           // ia 转换成指向数组首元素的指针
                        // ia[2] 得到 (ia + 2) 所指的元素
int *p = ia;             // p 指向 ia 的首元素
i = *(p + 2);            // 等价于 i = ia[2]
```

只要指针指向的是数组中的元素 (或者数组中尾元素的下一位置), 都可以执行下标运算:

```
int *p = &ia[2];         // p 指向索引为 2 的元素
int j = p[1];            // p[1] 等价于 *(p + 1), 就是 ia[3] 表示的那个元素
int k = p[-2];           // p[-2] 是 ia[0] 表示的那个元素
```

虽然标准库类型 `string` 和 `vector` 也能执行下标运算, 但是数组与它们相比还是有所不同。标准库类型限定使用的下标必须是无符号类型, 而内置的下标运算无此要求, 上面的最后一个例子很好地说明了这一点。内置的下标运算符可以处理负值, 当然, 结果地址必须指向原来的指针所指同一数组中的元素 (或是同一数组尾元素的下一位置)。



内置的下标运算符所用的索引值不是无符号类型, 这一点与 `vector` 和 `string` 不一样。

3.5.3 节练习

练习 3.34: 假定 $p1$ 和 $p2$ 指向同一个数组中的元素, 则下面程序的功能是什么? 什么情况下该程序是非法的?

```
p1 += p2 - p1;
```

练习 3.35: 编写一段程序, 利用指针将数组中的元素置为 0。

练习 3.36: 编写一段程序, 比较两个数组是否相等。再写一段程序, 比较两个 `vector` 对象是否相等。

3.5.4 C 风格字符串



尽管 C++ 支持 C 风格字符串，但在 C++ 程序中最好还是不要使用它们。这是因为 C 风格字符串不仅使用起来不太方便，而且极易引发程序漏洞，是诸多安全问题的根本原因。

字符串面值是一种通用结构的实例，这种结构即是 C++ 由 C 继承而来的 C 风格字符串 (C-style character string)。C 风格字符串不是一种类型，而是为了表达和使用字符串而形成的一种约定俗成的写法。按此习惯书写的字符串存放在字符数组中并以空字符结束 (null terminated)。以空字符结束的意思是在字符串最后一个字符后面跟着一个空字符 ('\\0')。一般利用指针来操作这些字符串。

C 标准库 String 函数

表 3.8 列举了 C 语言标准库提供的一组函数，这些函数可用于操作 C 风格字符串，它们定义在 cstring 头文件中，cstring 是 C 语言头文件 string.h 的 C++ 版本。

表 3.8: C 风格字符串的函数	
strlen(p)	返回 p 的长度，空字符不计算在内
strcmp(p1, p2)	比较 p1 和 p2 的相等性。如果 p1==p2，返回 0；如果 p1>p2，返回一个正值；如果 p1<p2，返回一个负值
strcat(p1, p2)	将 p2 附加到 p1 之后，返回 p1
strcpy(p1, p2)	将 p2 拷贝给 p1，返回 p1



表 3.8 所列的函数不负责验证其字符串参数。

传入此类函数的指针必须指向以空字符作为结束的数组：

```
char ca[] = {'C', '+', '+'};           // 不以空字符结束
cout << strlen(ca) << endl;           // 严重错误：ca 没有以空字符结束
```

此例中，ca 虽然也是一个字符数组但它不是以空字符作为结束的，因此上述程序将产生未定义的结果。strlen 函数将有可能沿着 ca 在内存中的位置不断向前寻找，直到遇到空字符才停下来。

比较字符串

比较两个 C 风格字符串的方法和之前学习过的比较标准库 string 对象的方法大相径庭。比较标准库 string 对象的时候，用的是普通的关系运算符和相等性运算符：

```
string s1 = "A string example";
string s2 = "A different string";
if (s1 < s2) // false: s2 小于 s1
```

如果把这些运算符用在两个 C 风格字符串上，实际比较的将是指针而非字符串本身：

```
const char ca1[] = "A string example";
const char ca2[] = "A different string";
if (ca1 < ca2) // 未定义的：试图比较两个无关地址
```

谨记之前介绍过的，当使用数组的时候其实真正用的是指向数组首元素的指针（参见 3.5.3 节，第 105 页）。因此，上面的 if 条件实际上比较的是两个 const char* 的值。这两个

指针指向的并非同一对象，所以将得到未定义的结果。

要想比较两个 C 风格字符串需要调用 `strcmp` 函数，此时比较的就不再是指针了。如果两个字符串相等，`strcmp` 返回 0；如果前面的字符串较大，返回正值；如果后面的字符串较大，返回负值：

```
if (strcmp(ca1, ca2) < 0) // 和两个 string 对象的比较 s1 < s2 效果一样
```

目标字符串的大小由调用者指定

连接或拷贝 C 风格字符串也与标准库 `string` 对象同类操作差别很大。例如，要想把刚刚定义的那两个 `string` 对象 `s1` 和 `s2` 连接起来，可以直接写成下面的形式：

```
// 将 largeStr 初始化成 s1、一个空格和 s2 的连接
string largeStr = s1 + " " + s2;
```

同样的操作如果放到 `ca1` 和 `ca2` 这两个数组身上就会产生错误了。表达式 `ca1 + ca2` 试图将两个指针相加，显然这样的操作没什么意义，也肯定是非法的。

正确的方法是使用 `strcat` 函数和 `strcpy` 函数。不过要想使用这两个函数，还必须提供一个用于存放结果字符串的数组，该数组必须足够大以便容纳下结果字符串及末尾的空字符。下面的代码虽然很常见，但是充满了安全风险，极易引发严重错误：

```
// 如果我们计算错了 largeStr 的大小将引发严重错误
strcpy(largeStr, ca1);           // 把 ca1 拷贝给 largeStr
strcat(largeStr, " ");           // 在 largeStr 的末尾加上一个空格
strcat(largeStr, ca2);           // 把 ca2 连接到 largeStr 后面
```

一个潜在的问题是，我们在估算 `largeStr` 所需的空间时不容易估准，而且 `largeStr` 所存的内容一旦改变，就必须重新检查其空间是否足够。不幸的是，这样的代码到处都是，程序员根本没法照顾周全。这类代码充满了风险而且经常导致严重的安全泄漏。



对大多数应用来说，使用标准库 `string` 要比使用 C 风格字符串更安全、更高效。

124

3.5.4 节练习

练习 3.37：下面的程序是何含义，程序的输出结果是什么？

```
const char ca[] = {'h', 'e', 'l', 'l', 'o'};
const char *cp = ca;
while (*cp) {
    cout << *cp << endl;
    ++cp;
}
```

练习 3.38：在本节中我们提到，将两个指针相加不但是非法的，而且也没什么意义。请问为什么两个指针相加没什么意义？

练习 3.39：编写一段程序，比较两个 `string` 对象。再编写一段程序，比较两个 C 风格字符串的内容。

练习 3.40：编写一段程序，定义两个字符数组并用字符串字面值初始化它们；接着再定义一个字符数组存放前两个数组连接后的结果。使用 `strcpy` 和 `strcat` 把前两个数组的内容拷贝到第三个数组中。

3.5.5 与旧代码的接口

很多 C++ 程序在标准库出现之前就已经写成了，它们肯定没用到 `string` 和 `vector` 类型。而且，有一些 C++ 程序实际上是与 C 语言或其他语言的接口程序，当然也无法使用 C++ 标准库。因此，现代的 C++ 程序不得不与那些充满了数组和/或 C 风格字符串的代码衔接，为了使这一工作简单易行，C++ 专门提供了一组功能。

混用 `string` 对象和 C 风格字符串



3.2.1 节（第 76 页）介绍过允许使用字符串字面值来初始化 `string` 对象：

```
string s("Hello World"); // s 的内容是 Hello World
```

更一般的情况是，任何出现字符串字面值的地方都可以用以空字符结束的字符数组来替代：

- 允许使用以空字符结束的字符数组来初始化 `string` 对象或为 `string` 对象赋值。
- 在 `string` 对象的加法运算中允许使用以空字符结束的字符数组作为其中一个运算对象（不能两个运算对象都是）；在 `string` 对象的复合赋值运算中允许使用以空字符结束的字符数组作为右侧的运算对象。

上述性质反过来就不成立了：如果程序的某处需要一个 C 风格字符串，无法直接用 `string` 对象来代替它。例如，不能用 `string` 对象直接初始化指向字符的指针。为了完成该功能，`string` 专门提供了一个名为 `c_str` 的成员函数：

```
char *str = s; // 错误：不能用 string 对象初始化 char*
const char *str = s.c_str(); // 正确
```

顾名思义，`c_str` 函数的返回值是一个 C 风格的字符串。也就是说，函数的返回结果是一个指针，该指针指向一个以空字符结束的字符数组，而这个数组所存的数据恰好与那个 `string` 对象的一样。结果指针的类型是 `const char*`，从而确保我们不会改变字符数组的内容。

125

我们无法保证 `c_str` 函数返回的数组一直有效，事实上，如果后续的操作改变了 `s` 的值就可能让之前返回的数组失去效用。



如果执行完 `c_str()` 函数后程序想一直都能使用其返回的数组，最好将该数组重新拷贝一份。

使用数组初始化 `vector` 对象

3.5.1 节（第 102 页）介绍过不允许使用一个数组为另一个内置类型的数组赋初值，也不允许使用 `vector` 对象初始化数组。相反的，允许使用数组来初始化 `vector` 对象。要实现这一目的，只需指明要拷贝区域的首元素地址和尾后地址就可以了：

```
int int_arr[] = {0, 1, 2, 3, 4, 5};
// ivec 有 6 个元素，分别是 int_arr 中对应元素的副本
vector<int> ivec(begin(int_arr), end(int_arr));
```

在上述代码中，用于创建 `ivec` 的两个指针实际上指明了用来初始化的值在数组 `int_arr` 中的位置，其中第二个指针应指向待拷贝区域尾元素的下一位置。此例中，使用标准库函数 `begin` 和 `end`（参见 3.5.3 节，第 106 页）来分别计算 `int_arr` 的首指针和尾后指针。在最终的结果中，`ivec` 将包含 6 个元素，它们的次序和值都与数组 `int_arr` 完全

一样。

用于初始化 `vector` 对象的值也可能仅是数组的一部分：

```
// 拷贝三个元素: int_arr[1]、int_arr[2]、int_arr[3]
vector<int> subVec(int_arr + 1, int_arr + 4);
```

这条初始化语句用 3 个元素创建了对象 `subVec`，3 个元素的值分别来自 `int_arr[1]`、`int_arr[2]` 和 `int_arr[3]`。

建议：尽量使用标准库类型而非数组

使用指针和数组很容易出错。一部分原因是概念上的问题：指针常用于底层操作，因此容易引发一些与烦琐细节有关的错误。其他问题则源于语法错误，特别是声明指针时的语法错误。

现代的 C++ 程序应当尽量使用 `vector` 和迭代器，避免使用内置数组和指针；应该尽量使用 `string`，避免使用 C 风格的基于数组的字符串。

3.5.5 节练习

练习 3.41：编写一段程序，用整型数组初始化一个 `vector` 对象。

练习 3.42：编写一段程序，将含有整数元素的 `vector` 对象拷贝给一个整型数组。



3.6 多维数组

严格来说，C++ 语言中没有多维数组，通常所说的多维数组其实是数组的数组。谨记这一点，对今后理解和使用多维数组大有益处。

当一个数组的元素仍然是数组时，通常使用两个维度来定义它：一个维度表示数组本身大小，另外一个维度表示其元素（也是数组）大小：

```
int ia[3][4]; // 大小为 3 的数组，每个元素是含有 4 个整数的数组
// 大小为 10 的数组，它的每个元素都是大小为 20 的数组，
// 这些数组的元素是含有 30 个整数的数组
int arr[10][20][30] = {0}; // 将所有元素初始化为 0
```

如 3.5.1 节（第 103 页）所介绍的，按照由内而外的顺序阅读此类定义有助于更好地理解其真实含义。在第一条语句中，我们定义的名字是 `ia`，显然 `ia` 是一个含有 3 个元素的数组。接着观察右边发现，`ia` 的元素也有自己的维度，所以 `ia` 的元素本身又都是含有 4 个元素的数组。再观察左边知道，真正存储的元素是整数。因此最后可以明确第一条语句的含义：它定义了一个大小为 3 的数组，该数组的每个元素都是含有 4 个整数的数组。

使用同样的方式理解 `arr` 的定义。首先 `arr` 是一个大小为 10 的数组，它的每个元素都是大小为 20 的数组，这些数组的元素又都是含有 30 个整数的数组。实际上，定义数组时对下标运算符的数量并没有限制，因此只要愿意就可以定义这样一个数组：它的元素还是数组，下一级数组的元素还是数组，再下一级数组的元素还是数组，以此类推。

对于二维数组来说，常把第一个维度称作行，第二个维度称作列。